Node.js — Day 8

File System (fs): Checking & Deleting Files

Learning goal: Learn how to check whether a file exists and how to delete a file using Node.js's built-in **fs** module.

> **Previous knowledge:** Day 7 covered **fs.readFile()** and **fs.writeFile()**, including asynchronous file operations and their connection to the Event Loop.

1. Checking Whether a File Exists

fs.existsSync() checks whether a file or path exists and returns a Boolean value.

```
const fs = require("fs");

const exists = fs.existsSync("./message.txt");

console.log(exists);
```

If the file exists, the result is:

```
true
```

If it does not exist, the result is:

```
false
```

> **Mental model:**

```
existsSync()
     ↓
Does it exist?
     ↓
true / false
```

2. What Does Sync Mean Here?

The **Sync** in **existsSync()** means this particular operation is **synchronous**. It returns the answer immediately and blocks execution while it performs the check.

For Day 8, the key point is simply: **existsSync() is synchronous**.

3. Deleting a File with fs.unlink()

fs.unlink() is used to delete a file. It is asynchronous.

```
const fs = require("fs");

fs.unlink("./message.txt", (err) => {
  if (err) {
    console.log(err);
    return;
  }

  console.log("File deleted successfully");
});
```

> If **message.txt** exists, Node.js deletes it and the success message is printed.

4. Error Handling with unlink()

If the file does not exist, **unlink()** produces an error. The callback therefore checks **err** before reporting success.

```
if (err) {
  console.log(err);
  return;
}
```

The **return** stops the callback from continuing into the success message when an error occurred.

5. Your fs Toolkit So Far

Method	Purpose	Type learned
fs.readFile()	Read file contents	Asynchronous
fs.writeFile()	Write / replace file contents	Asynchronous
fs.unlink()	Delete a file	Asynchronous
fs.existsSync()	Check whether a file/path exists	Synchronous

6. Practical Mental Model

```
readFile()
     ↓
   READ

writeFile()
     ↓
   WRITE / REPLACE

unlink()
     ↓
   DELETE

existsSync()
     ↓
   CHECK
```

7. Important Rules to Remember

• **fs** means File System and is built into Node.js.

• **existsSync()** checks whether a file/path exists and returns **true** or **false**.

• **existsSync()** is synchronous.

• **unlink()** deletes a file.

• **unlink()** is asynchronous.

• If **unlink()** cannot delete the file, the callback receives an error.

• Check **err** before treating an asynchronous operation as successful.

8. Day 8 Examples

Example 1 — Check a file

```
const fs = require("fs");

console.log(fs.existsSync("./message.txt"));
```

Result: **true** if the file exists; otherwise **false**.

Example 2 — Delete a file

```
const fs = require("fs");

fs.unlink("./old.txt", (err) => {
  if (err) {
    console.log(err);
    return;
  }

  console.log("Deleted");
});
```

If successful, **old.txt** is deleted and **Deleted** is printed.

9. Day 8 Quick Recap

existsSync() → CHECK → true / false

unlink() → DELETE → asynchronous

err → tells us an asynchronous operation failed

We now know four useful file-system operations: **read, write, delete, and check**.

10. Key Takeaway

> **The fs module gives Node.js applications the ability to work with files. By Day 8, you can read, write, check, and delete files, and you understand that different fs methods can be synchronous or asynchronous.**

Practice Completed

Day 8: **2 / 2 questions correct** ■

Day 8 is complete. No additional topic was added beyond what was taught in the lesson.